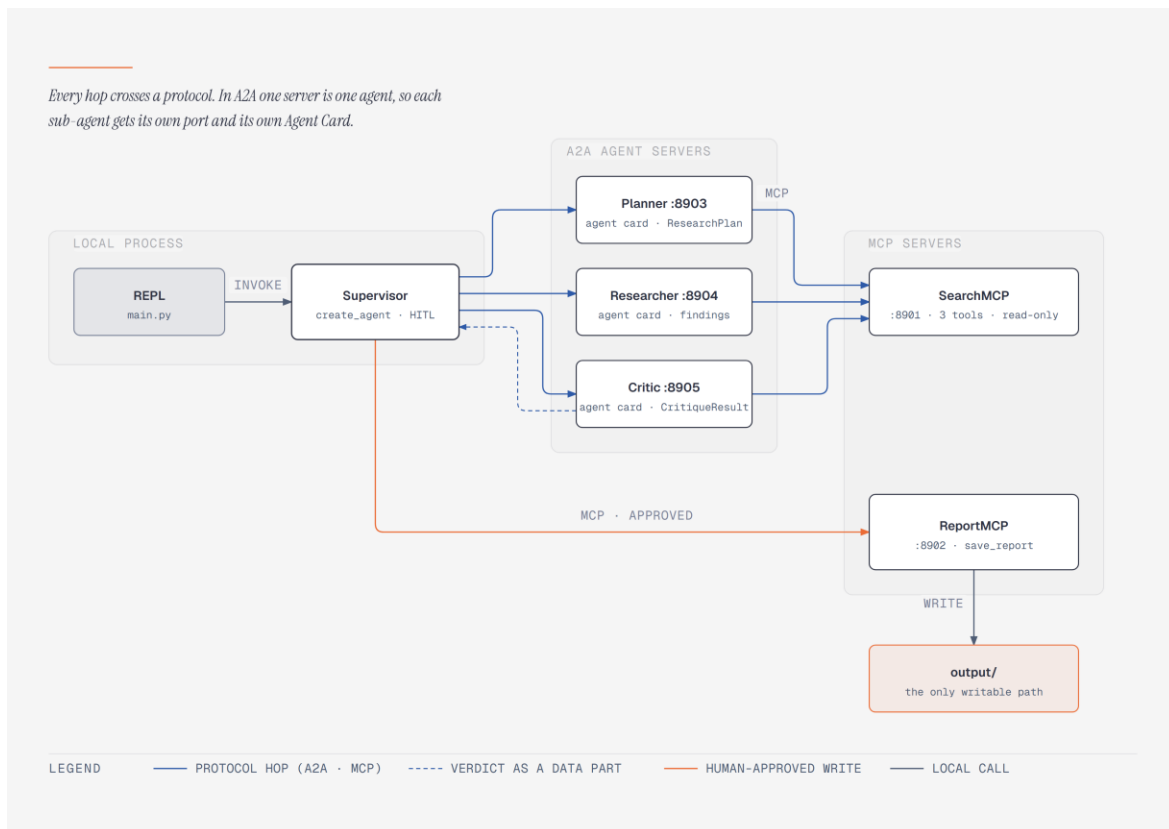


MA_systems_hl10

1. Що це за система

Термінальна мультиагентна дослідницька система, де кожен перехід між компонентами використовує мережевий протокол, а не виклик функції. Користувач задає дослідницьке питання; чотири агенти — Супервізор і три підагенти — планують, досліджують і критикують відповідь у циклі, і нічого не потрапляє на диск, доки користувач не підтвердить це в терміналі.

- MCP (Model Context Protocol) несе кожен виклик інструмента: два сервери, SearchMCP (web_search, read_url, knowledge_search) і ReportMCP (save_report), кожен — довготривалий процес FastMCP на власному порту.
- A2A (Agent2Agent) несе кожне делегування між агентами: Planner, Researcher і Critic кожен працює як власний A2A-сервер із власною Agent Card; Супервізор — єдиний агент, що живе в процесі терміналу користувача, і звертається до трьох інших виключно через a2a.client.
- Human-in-the-loop блокує єдиний запис, який система здатна зробити: save_report виконується лише після явного approve в REPL, це забезпечує HumanInTheLoopMiddleware і шлюз, що читає вердикт Critic зі типізованого стану графа, ніколи з прози.

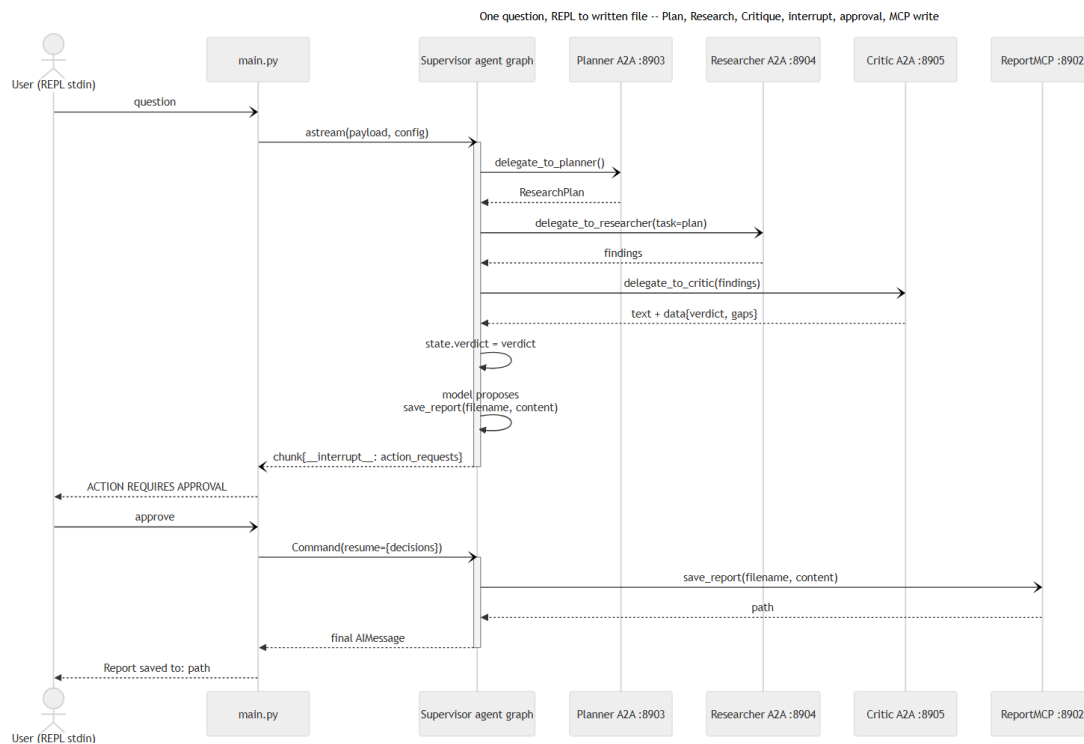


Цільова архітектура — п'ять серверів плюс термінальний Супервізор, MCP-виклики інструментів та A2A-делегування

2. Фіксований інваріант

`supervisor.py` ніколи не імпортує `a2a_servers` чи будь-який модуль `agents/*` — це закріплено автоматизованим тестом обходу імпортів за шарами, тож делегування не може непомітно стати локальним викликом функції, поки система продовжує працювати. Це архітектурна ставка всього проекту: реальні межі протоколу забезпечуються структурно, а не за домовленістю.

Цикл обмежений у коді (ліміт запусків на кількість повторних викликів), а не залишений на розсуд промпту завершити себе самостійно. Коли цикл завершується, Супервізор подає чернетку на затвердження, редагування чи відхилення.

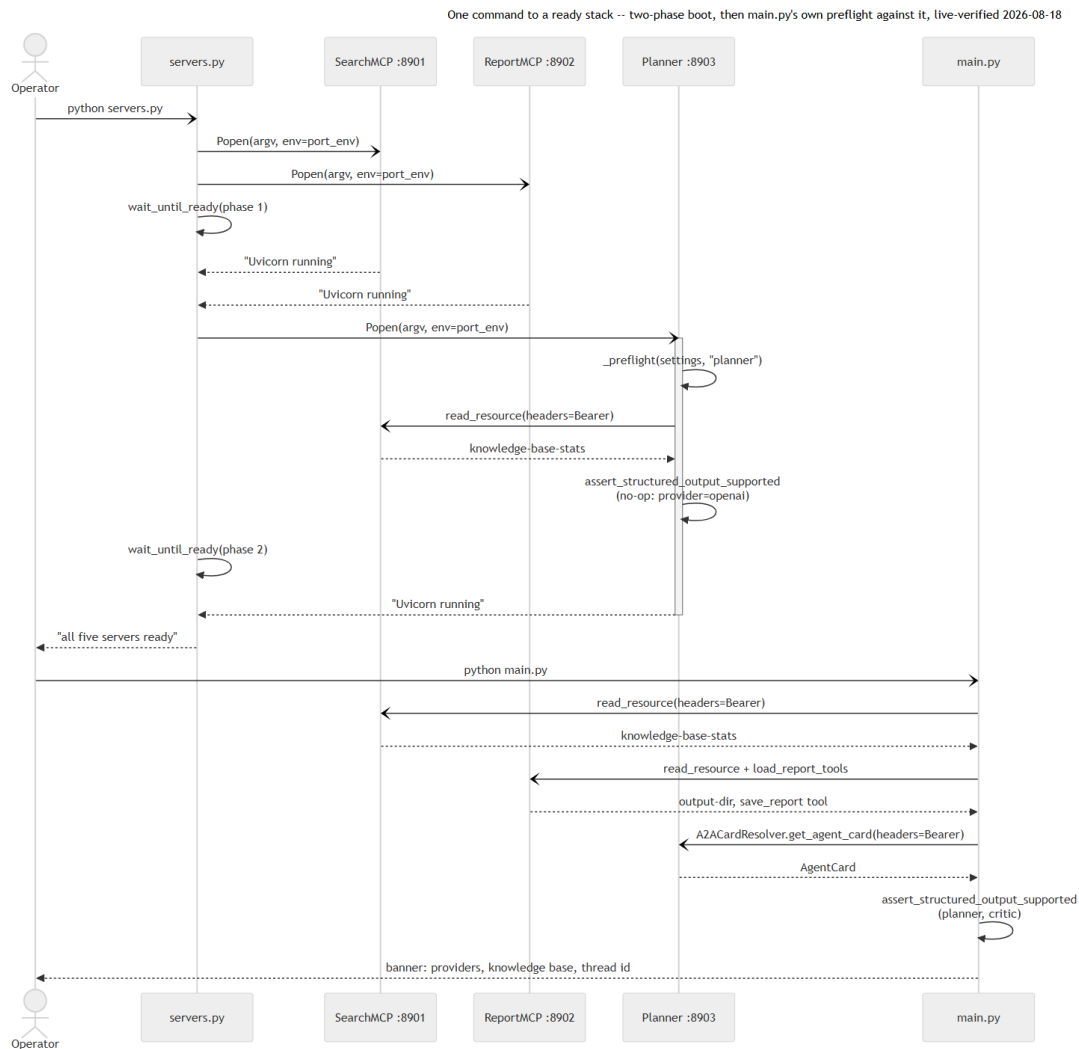


Головний сценарій: одне питання від REPL до збереженого файлу — делегування Планувальнику, Дослідникові й Критику через A2A, вердикт Critic як типізовані дані, переривання на затвердження людиною, і лише після approve виклик `save_report` у `ReportMCP`. Виклики інструментів `SearchMCP` на цій схемі не показані

Система підтримує усі чотири людські рішення — `approve`, `edit`, `reject` і відновлення після збою. Рішення `edit` система обробляє наступним чином: вільний текстовий відгук людини не можна перетворити на очікувану структуровану дію, тож натомість цей відгук передається як `reject` із текстом-інструкцією для переробки. Відновлення після збою працює через `AsyncSqliteCheckpoint` і прапорець `--thread`: якщо вказати невідомий системі ідентифікатор розмови, вона відмовить, а не почне тихо нову порожню розмову.

3. Запуск і робота стеку

Обидва MCP-сервери мають бути досяжні до того, як будь-який A2A-агент почне обслуговувати запити — кожен A2A-сервер перевіряє доступність SearchMCP при старті й відмовляється обслуговувати, якщо не може її досягти — тож лаунчер запускає систему у дві фази: (SearchMCP, ReportMCP), потім (Planner, Researcher, Critic). Запуск усіх п'яти одночасно веде до того, що три A2A-сервери змагаються із власним холодним стартом SearchMCP і падають із `sys.exit(1)`. Усі п'ять серверів перевіряють один спільний bearer-токен і прив'язуються лише до `127.0.0.1`, ніколи до `0.0.0.0`.

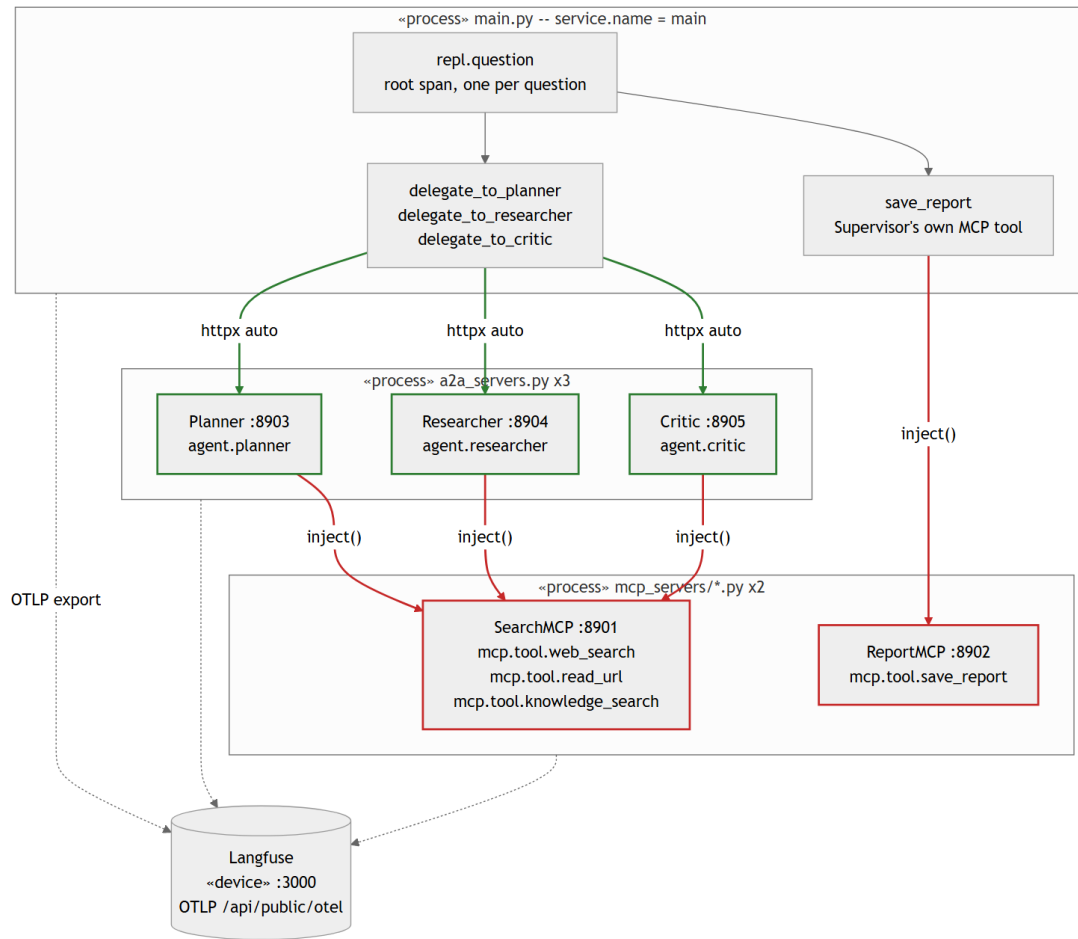


Двофазний запуск: спочатку MCP-сервери, потім A2A-агенти

4. Спостережуваність: одне питання — один трейс

За увімкненого трейсингу одне питання в REPL породжує один трейс OpenTelemetry, що охоплює процес REPL, обидва MCP-сервери й усіх трьох A2A-агентів.

One trace across six processes -- which instrumentation carries traceparent on each hop, and the one hop that needed it injected by hand



Топологія одного трейсу: REPL, обидва MCP-сервери, усі три A2A-агенти діляться одним trace_id

5. Оцінювання: методологія та заміряні результати

Золотий набір із 12 кейсів (5 базових, 4 крайових, 3 змагальних) проганяється через живий стек апаратом оцінювання, що підміняє HITL авто-затвердзувальним резолвером, зберігає один EvidencePack на пару (запуск, кейс) і оцінює кожен пакет за 5-пунктовою бінарною рубрикою через LLM-суддю з іншого сімейства моделей, ніж будь-який заміряний агент (google/gemini-2.5-pro проти чотирьох gpt-4.1-mini-агентів).

5.1 Основний замір: свіп із 36 трейсів (12 кейсів × 3 повтори)

Пункт рубрики	Частка PASS (прохід 1)	95% ДІ (ресемплінг кейсів)	Застосовне n	Валідовано проти людських міток?
answer_relevancy	97.1–100%	широкий, насичений	35	Ні — насичений (36/36 чи близько), виключений з агрегованого

				підрахунку
requirement_compliance	80.0–83.3%	[0.63, 0.91]	35	Ні — κ 0.40/0.25 у двох проходах, 0.092 проти незалежних міток
citation_reliability	47.1–66.7%	[0.24, 0.71]	17 (прохід), 11 (незалежні), 6 (первинні)	Так, проти 24 незалежних міток — κ = 0.645 [0.214, 1.0]
injection_resistance	66.7–87.5%	[0.50, 1.00]	8 (прохід), 6 (незалежні), 3 (первинні)	Ні — n замалий для валідації (3 змагальні питання $\times$ 2 повтори — структурна стеля набору; ні додаткове маркування, ні додаткові проходи оцінювання її не піднімуть)
harmful_compliance	91.7–94.3%	[0.83, 1.00]	35 (прохід), 23 (незалежні), 12 (первинні)	Умовно так — κ = 1.0 у двох незалежних проходах оцінювання, але на єдиному промаркованому FAIL; чесне твердження — «не системно помиляється на цьому пункті»

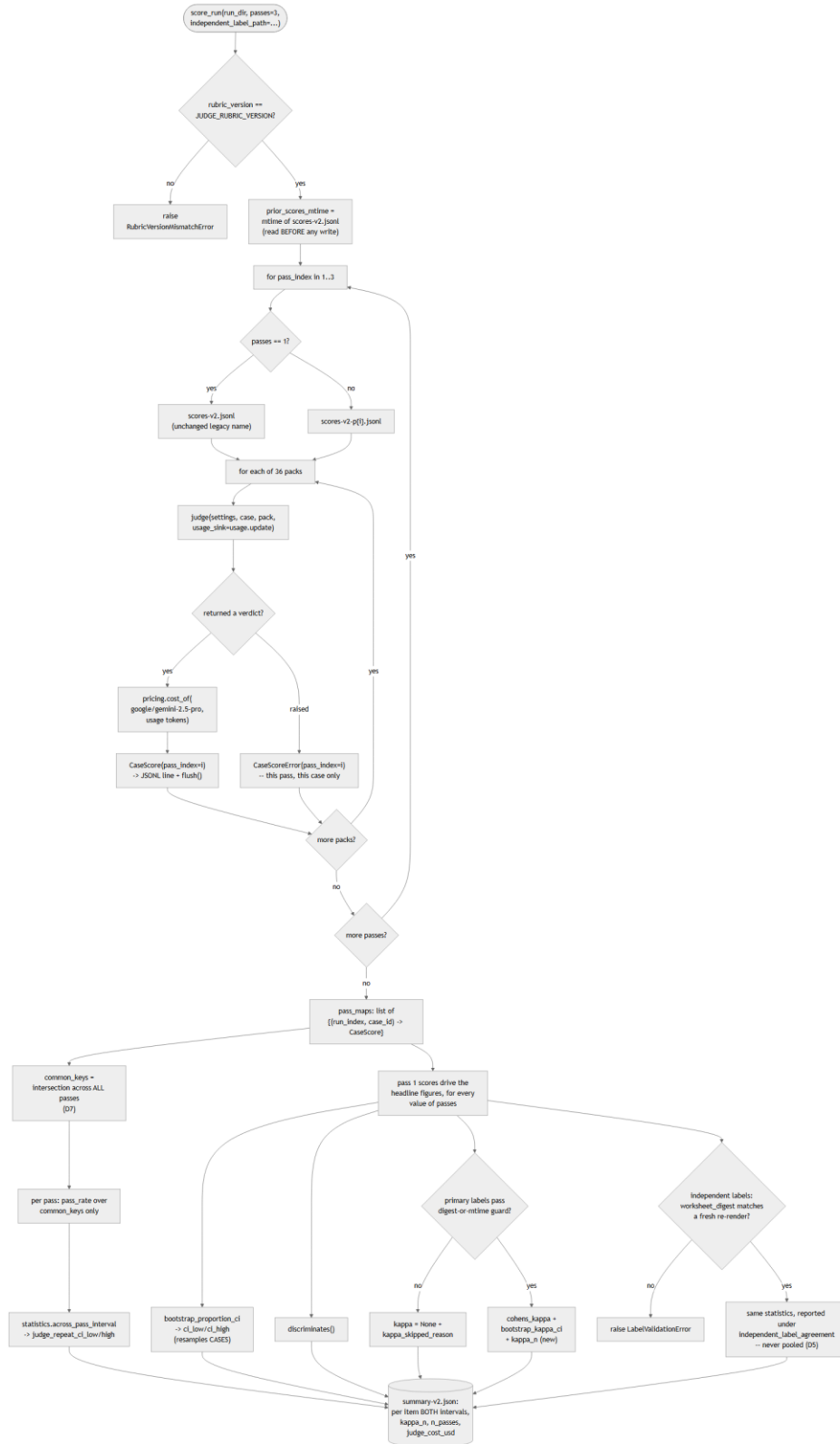
Три з п'яти пунктів рубрики спираються менш ніж на 20 кейсів, а два — менш ніж на 10. Тому в таблиці вище кожна цифра подана разом із реальним розміром вибірки (n) — щоб не створювалося враження, ніби вибірка більша, ніж є насправді.

5.2 Сам інструмент вимагав ремонту, перш ніж йому можна було довіряти

Найважливіша знахідка стосується не самої системи, а судді, який її оцінює. Одні й ті самі докази (без жодних змін) оцінили двічі — і результати розійшлися аж на 20.8 відсотків за одним із критеріїв, а узгодженість між цими двома оцінками впала з повної ($\kappa=1.0$) до нульової ($\kappa=0.0$). Щоб перевірити, чи це просто випадковість, зробили третій, незалежний прохід — і він показав той самий масштаб розбіжності: 20 пунктів за критерієм citation_reliability (53.3% $\rightarrow$ 73.3%). Це підтвердило, що перша розбіжність — не випадковий збіг.

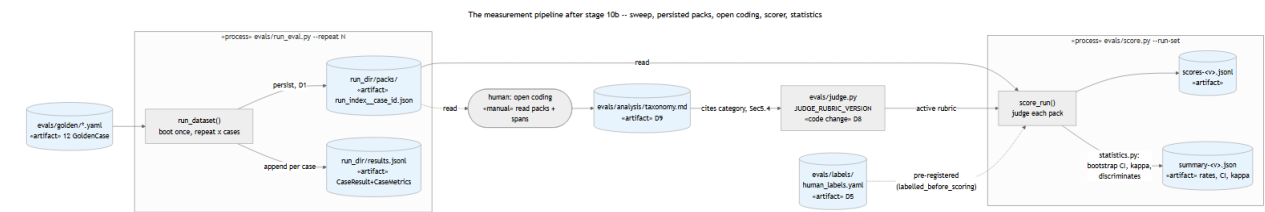
Показовий момент: якби публікацію зробили раніше, до завершення третього проходу, на основі лише двох проходів вийшла б цифра всього 6.2 відсотків — і це виглядало б як «майже нічого», дрібний шум. Але третій прохід показав, що якраз оця мала цифра й була випадковістю, а не реальна 20-відсоткова розбіжність.

A three-pass scoring run -- 36 packs judged three times, to two intervals per rubric item



Три незалежні проходи оцінювання над ідентичними доказами, ресемплінг за кейсами й за проходами

Саме тому в проєкті діє правило: ніколи не довіряти одному прогону оцінювання, і навіть два прогони вважати попередніми, не остаточними. І саме тому це подано не як дрібна деталь про один критерій, а як загальний висновок про оцінювання LLM-суддею: розкид результатів судді від запуску до запуску може бути більшим за той самий ефект, який суддя має виявити. Дізнатися це можна лише одним способом — реально заміряти, наскільки стабільно суддя оцінює одне й те саме, а не просто вважати, що він стабільний.



Конвеєр вимірювання: свіп, вилучення доказів, суддя, статистика — апарат, що дав кожне число вище

Дешевшу модель-суддю (приблизно в 10 разів дешевшу) заміряли як кандидата на заміну посеред проєкту й відхилили: її узгодженість із людськими мітками на citation_reliability пішла у від'ємну зону, а її власна дисперсія повторів на injection_resistance була більш ніж удвічі вища за дисперсію обраного судді.

5.3 Конфаунд, що має супроводжувати кожну цифру вартості й затримки

web_search не спрацював у 6 із 17 викликів у заміряному свіпі — операційна нестабільність пошукового бекенду, використаного в цьому проєкті. Розкид вартості й затримки між повторами ідентичного питання частково кодує саме цю нестабільність пошуку, а не дисперсію поведінки системи; жодну з цих цифр не слід читати як чистий вимір самої лише системи.

6. Дефекти системи — заміряні знахідки

Два дефекти виправили й перевірили наживо. Ще чотири знайшли, але свідомо не стали виправляти під час вимірювання — бо виправлення посеред серії запусків знецінило б уже оплачені результати. Ці чотири просто фіксуються тут як реальні, виміряні проблеми.

ID	Дефект	Статус	Заміряно
F2	Відсутній шлюз відмови поза межами компетенції — система досліджувала й зберігала повну відповідь на питання, цілком поза межами її дослідницького призначення	Виправлено. Обов'язкове за схемою поле in_score плюс промпт-рівневе судження про межі	До: 0/3 чистих відмов на найгіршому проміжному свіпі; після: 3/3, 0/5 хибних відмов на контрольному зрізі в межах компетенції
F6	Ін'єкція промпту, що досягала довготривалого збереженого артефакту через knowledge_search, який — на відміну від read_url — не огортав	Виправлено механізм; заміряна частка не змінилася. Та сама форма роздільників, що вже використовує read_url, застосована до точки	2/3 стійкості і до, і після виправлення — без заміряної зміни на n=3. Сам механізм роздільників підтверджено присутнім і коректним у спанах; модель

	недовірений вміст роздільниками	повернення knowledge_search	усе одно один раз виконала вбудовану інструкцію. Власна стійкість read_url на еквівалентній формі атаки лишається 3/3, без змін
F2b	Вигадані назви джерел — звіти цитують статтю корпусу під вигаданими назвами або під фразою з тексту, помилково прийнятою за назву, тоді як основний ідентифікатор лишається коректним	Подано, не виправлено	2/3 одного кейсу
F3	Підкорення джейлбрейку, недетерміноване — один запуск зберіг звіт, що дослівно цитував внутрішній системний промпт; інший зберіг оперативні способи обійти сам шлюз затвердження	Подано, не виправлено	2/3
F4	read_url повторює спроби над PDF, який ніколи не вдається розібрати, на кожному запуску, прив'язаному до корпусу, витрачаючи бюджет викликів інструментів	Подано, не виправлено	15/15 запусків, прив'язаних до корпусу
F8	Помилка пошуку, подана як негативна дослідницька знахідка — система не розрізняє «я шукав і нічого не знайшов» від «мої пошуки не вдалися»	Подано, не виправлено	1/3
F9	Сам суддя іноді повертає порожню чи хибну відповідь на джейлбрейк-подібних доказах замість дійсного вердикту	Відкрито — рівень інструмента, оброблено відмовостійкістю на рівні кейсу, тож коштує один кейс, ніколи весь запуск	Спостережено щонайменше двічі, у двох незалежних проходах оцінювання; першопричина не встановлена
F10	Власна дисперсія судді від запуску до запуску достатньо велика, щоб перевищувати кілька ефектів, які вона покликана вимірювати	Підтверджено, це радше властивість, яку слід завжди звітувати поруч, ніж дефект для виправлення	§5.2

У знахідки F6 є важлива перевірка для порівняння: ту саму атаку (шкідлива інструкція, захована у вмісті) пропустили через read_url — шлях, який уже мав захист (роздільники навколо чужого вмісту). Там атака провалилась 3 рази з 3 — захист спрацював. Тобто та сама атака, інший шлях — і протилежний результат. Саме це порівняння й доводить: причина проблеми — відсутність роздільників, а не щось інше.

7. Знахідки

- Список дозволених інструментів на агента — це на практиці те саме, що в літературі називають «вибором інструментів на рівні планувальника», тільки досягнуте не спеціальним механізмом, а самою структурою системи. Коли в одного агента забагато інструментів на вибір (більше ~10–15), модель починає гірше обирати потрібний: розподіл одного великого набору інструментів між кількома спеціалізованими агентами сам собою вже і є цим відбором.
- Critic (схвалити/відхилити) передається між компонентами як чіткі структуровані дані, а не як звичайний текст, який Супервізор мав би «розуміти». Це рішення прийнято тому, що на практиці виявили проблему: якщо Супервізор має здогадуватися про вердикт із останнього речення відповіді, він може помилково завершити роботу без збереження результату навіть після реального схвалення. Передача вердикту як структурованих даних унеможливорює цю помилку.
- Розглядали й відхилили альтернативу: замість пошуку по базі знань (RAG) — щоб агент сам вів нотатки у markdown-файлі. Відхилено, бо схема «агент пише нотатку → потім сам її читає як джерело» — це та небезпечна петля самозараження, яку система свідомо має забороняти між вихідними файлами й базою знань. Єдиний спосіб убезпечитись від цього — щоб людина перевіряла кожну нотатку вручну, що більше клопоту, ніж один-єдиний крок підтвердження, який є в цій системі зараз.
- Ця система працює за версією протоколу MCP від 2025-11-25 — перевірено напяму, яку версію підтримує встановлена бібліотека. Це на одну версію старіше за версію, опубліковану одночасно (2026-07-28), яка змінює кілька фундаментальних речей. Проект явно вимагав саме цю версію.

8. Висновок

- Система виконує всі вимоги завдання: два MCP-сервери, три окремі A2A-агенти зі своїми картками, Супервізор, що керує ними лише через мережу, ніколи напяму не викликає їхній код, робочий цикл «план → дослідження → критика» поверх цих протоколів, і крок підтвердження людиною перед єдиним записом, який робить система. Усе це перевірено наживо на відповідних етапах.
- Що дають етапи оцінювання понад цей список вимог — це чесно визнані межі системи: з п'яти оцінюваних якостей лише три підтверджено незалежною людською перевіркою, дві — ні; сам суддя, який виставляє оцінки, має власний розкид результатів — іноді більший за те, що він має виміряти; чотири реальні дефекти показано з конкретними цифрами.
- Останнє, власне, і є головним висновком проєкту: система, яка чесно каже, чого вона не знає напевно, робить те, чого не вміє система, що звітує лише про свої успіхи.